LAPORAN PRAKTIKUM MIKROPROSESOR DAN INTERFACE

PERKALIAN BILANGAN DESIMAL

Disusun untuk memenuhi tugas praktikum Mikroprosesor Dan Interface

Oleh

Gaitsha Azzahra Devitrya

231344012



Tanggal Percobaan: 2 Mei 2025

Tanggal Penyerahan: 9 Mei 2025

Instruktur: 1. Ferry Satria, BSEE., M.T.

2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2025

DAFTAR ISI

DAFTAR ISI.....ii
A. TUJUAN.....2
B. LANDASAN TEORI.....2
C. ALAT DAN BAHAN.....4
D. LANGKAH KERJA.....5
E. HASIL PERCOBAAN DAN ANALISIS DATA.....6
G. KESIMPULAN.....25
DAFTAR PUSTAKA.....26

A. TUJUAN

1. Mahasiswa dapat memahami konsep perkalian data bilangan desimal
2. Mahasiswa dapat membuat program pada EMU8086 untuk menyelesaikan operasi perkalian bilangan desimal.

B. LANDASAN TEORI

A. Definisi microprosesor

Mikroprosesor dikenal juga sebagai CPU (Central Processing Unit). CPU terdiri dari ALU, Control Unit, Register , Bus System, dan Clock untuk melakukan komputasi.

B. Register

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Terdapat dua jenis register yang dibedakan menurut besar Bit, yaitu Register 8-Bit dan Register 16-Bit.

Register 8-Bit hanya memiliki 1 bagian jenis register yaitu Register umum : AH, AL, BH, BL, CH, CL, DH, DL

Dan Register 16-Bit memiliki 5 bagian jenis register yang memiliki tugasnya masing-masing, yaitu :

1. Segmen Register.

Register yang termasuk dalam kelompok ini terdiri atas register CS (Code Segment), DS (Data Segment), ES (Extra Segment) dan SS (Stack Segment) yang masing-masingnya merupakan register 16 bit. Register-register dalam kelompok ini secara umum digunakan untuk menunjukkan alamat dari suatu segmen.

2. General Register (Register Umum)

Register yang termasuk dalam kelompok ini adalah register AX, BX, CX dan DX yang masing-masing terdiri atas 16 bit. Register- register 16 bit dari kelompok ini mempunyai suatu ciri khas, yaitu dapat dipisah menjadi 2 bagian dimana masing-masing bagian terdiri atas 8 bit, Akhiran H menunjukkan High sedangkan akhiran L menunjukkan Low.

3. Register Pointer

Register ini menunjukkan alamat sebuah data di lokasi memori, dipakai saat operasi perpindahan data(dari/ke memori) dan penunjukkan alamat suatu instruksi. Jenis-jenis register pointer yaitu BP (Base Pointer) dan IP (Instruction Pointer).

4. Register Index

Sama dengan pointer register, sering digunakan untuk menunjukkan alamat sebuah data di lokasi memori pada operasi string. Jenisnya ada SI (Source Index), DI (Destination Index).

5. Register Stack

Register Stack (SP) atau Stack Pointer adalah salah satu jenis register dalam arsitektur komputer yang digunakan untuk menunjuk alamat teratas dari stack dalam memori.

6. Register Flag [REG 16 BIT]

Register Flag berfungsi untuk menunjukkan status(keadaan) sesaat dari mikroprosesor. Bit-bit pada flag akan mengalami perubahan, tergantung proses yang berlangsung. Sesuai dengan namanya Flags(Bendera) register ini menunjukkan kondisi dari suatu keadaan< ya atau tidak >. Karena setiap keadaan dapat digunakan 1 bit saja, maka sesuai dengan jumlah bitnya, Flags register ini mampu memcatat sampai 16 keadaan.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	OF	DF	IF	TF	SF	ZF	-	AF	-	PF	-	CF

C. Syntax/Perintah Mikroprosesor 8086

1. ADC: Penjumlahan dengan Carry
2. ADD: Penjumlahan
3. CMP : Membandingkan
   Memiliki Algoritma: Operand 1 – Operand 2
   Hasil tidak disimpan dimana-mana, hanya saja mempengaruhi Flag (OF, SF,ZF, AF, PF, CF) berdasarkan hasil.
4. MOV : Instruksi MOV tidak bisa dilaksanakan jika:
   - Memberikan nilai ke Register CS dan Register IP
   - Tidak bisa Menyalin nilai dari satu segmen ke segmen Register yang lain (Seharusnya menggunakan Register umum terlebih dahulu)
   - Menyalin segera data ke segmen register (Seharusnya menggunakan Register umum terlebih dahulu)
5. INC : Menaikkan
   Memiliki Algoritma : Operand = Operand + 1

6. JNZ(Jump if Not Zero) = untuk melakukan lompatan ke alamat tertentu jika hasil operasi sebelumnya tidak nol.
7. DUP : untuk pendeklarasian variabel di segmen .DATA untuk mengalokasikan memori dalam jumlah tertentu dengan nilai awal yang sama.
8. DAT bukan instruksi bawaan Assembly, yang digunakan adalah DB (Define Byte), DW (Define Word), atau DD (Define Double Word) untuk mendefinisikan data.
   - DB (Define Byte) → Menyimpan 1 byte data
   - DW (Define Word) → Menyimpan 2 byte data
   - DD (Define Double Word) → Menyimpan 4 byte data

D. Perintah DAA (Decimal Adjust after Addition)

Perintah DAA (Decimal Adjust after Addition) digunakan untuk menyesuaikan hasil penjumlahan agar tetap dalam format BCD (Binary-Coded Decimal) setelah operasi penambahan. Instruksi ini bekerja dengan memeriksa nilai di register AL setelah operasi penjumlahan biasa menggunakan ADD atau ADC**.** Jika bagian rendah dari register AL (4 bit paling rendah) lebih besar dari 9 atau jika AF (Auxiliary Carry Flag) diatur, maka mikroprosesor akan menambahkan 6 (06H) ke bagian rendah AL untuk memastikan hasil tetap dalam format BCD. Demikian pula, jika bagian tinggi AL (4 bit paling atas) lebih besar dari 9 atau jika CF (Carry Flag) diatur, maka mikroprosesor akan menambahkan 60H ke AL dan mengatur CF sebagai tanda bahwa ada carry ke digit berikutnya. Perintah DAA umumnya digunakan dalam aplikasi yang menangani angka desimal.

E. Subrutin pada mikroprosesor

Subrutin pada mikroprosesor emu8086 adalah bagian dari program yang dirancang untuk menjalankan tugas tertentu dan dapat dipanggil dari bagian lain dalam program utama. Subrutin digunakan untuk mengurangi redundansi kode, meningkatkan keterbacaan, serta mempermudah pemeliharaan program. Dalam assembly 8086, subrutin dideklarasikan sebagai procedure (PROC) dan diakhiri dengan RET untuk kembali ke pemanggilnya.

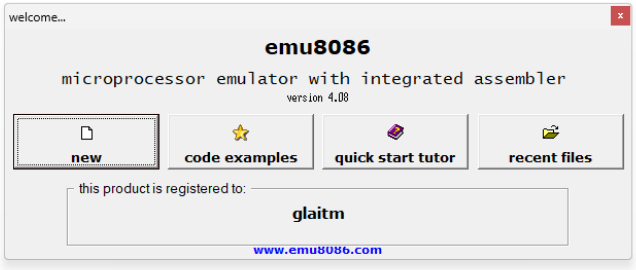
Ketika sebuah subrutin dipanggil menggunakan instruksi CALL**,** alamat instruksi berikutnya akan disimpan di stack agar setelah eksekusi subrutin selesai, program dapat kembali ke titik pemanggilan dengan menggunakan instruksi RET. Proses ini memungkinkan penggunaan kembali kode tanpa perlu menuliskan instruksi yang sama berulang kali.

C. ALAT DAN BAHAN

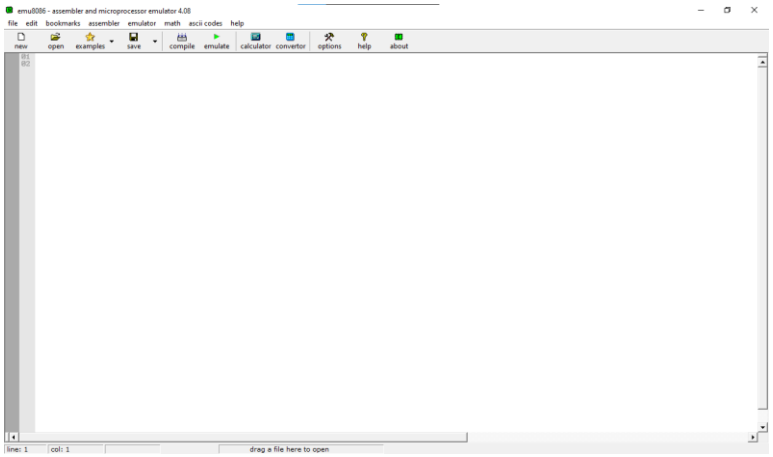
1. Laptop/PC
2. Software EMU 8086

D. LANGKAH KERJA

1. Instal Software emu8086 pada laptop dan buka aplikasinya
2. Setelah membuka aplikasi maka akan muncul tampilan *new* pada monitor

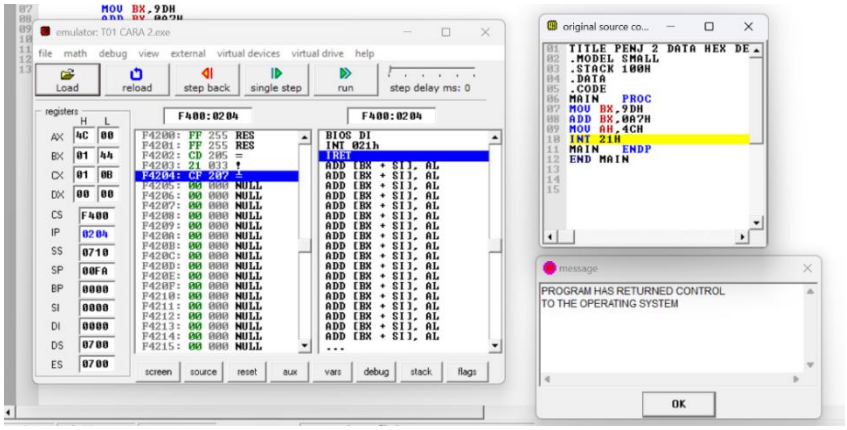


3. Maka setelah itu akan tampil worksheet seperti gambar dibawah ini :



Worksheet ini digunakan untuk menuliskan kode.

4. Untuk memeriksa kode yang berada di worksheet apakah program sudah benar atau belum, klik “compile”
5. Jika sudah disave maka jika program sudah benar akan ada tulisan “run”.
6. Jika sudah di ”run” maka akan muncul hasilnya pada monitor



E. HASIL PERCOBAAN DAN ANALISIS DATA

Pada praktikum ini, menggunakan aplikasi emu8086 dimana akan menghitung perkalian bilangan desimal. Perkalian dilakukan dengan menggunakan metode penjumlahan berulang, pada programnya dapat menggunakan instruksi aritmetika seperti ADD atau ADC. Namun, hasil penjumlahan bisa keluar dari format BCD, sehingga perlu dikoreksi menggunakan instruksi DAA (Decimal Adjust after Addition) valid dalam sistem desimal. Dalam implementasi, angka-angka BCD biasanya disimpan di memori, kemudian diakses menggunakan indeks (SI atau DI) dalam sebuah loop untuk dijumlahkan satu per satu.

Kemudian pada praktikum kali ini juga, terdapat bagian tambahan pada program yang disebut dengan subrutin, bagian ini digunakan pada program jika hasil penjumlahan yang akan ditampilkan dalam bentuk Binary Coded Decimal(BCD) masih belum benar. Subrutin ini mengecek setiap digit satuan dan puluhan, lalu menambahkan 6 jika diperlukan untuk mempertahankan format BCD.

Kode program subrutin diperlukan pada praktikum ini, karena pada praktikum ini hampir semuanya merupakan penjumlahan yang akan ditampilkan dalam format BCD.Kode subrutin yang digunakan pada praktikum kali ini adalah sebagai berikut :

```
SDAA PROC   ; SUBRUTIN SDAA
     PUSH BX
     PUSH CX
     MOV BH,AH
     LAHF
     MOV BL,AH
     MOV CH,AL
     MOV CL,AL
     AND CL, 0FH; CL=SATUAN
     ;===============
     TEST AH,10H
     JZ L22
     ADD AL,6
     JNC L22
     ADD AL,60H
     OR BL,1
L22: TEST AH,1
     JZ L33
     ADD AL,60H
     OR BL,1
L33: MOV CH,AL
     AND CH,0FH
     CMP CH,9
     JBE L44
     ADD AL,6
     JNC L44
     ADD AL,60H
     OR BL,1
L44: MOV CH,AL
     AND CH,0F0H
     CMP CH,90H
     JBE L55
     ADD AL,60H
     OR BL,1
L55: MOV AH,BL
     SAHF
     POP CX
     POP BX
     MOV AH,BH
     RET
SDAA ENDP
END MAIN
```

Pada praktikum kali ini terdapat 5 soal yang berhasil dikerjakan yang dapat dianalisis sebagai berikut :

1. Buat program untuk mengalikan data desimal 4 digit yang ada pada register BX dengan data desimal 2 digit yang ada pada register CH. Simpan hasil perkalian pada register CL:DX

   Program :

```
TITLE SOAL 1 ;MENGALIKAN 4 DIG Dengan 2 DIG BIL DES DI (
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC

        ADD BX,0    ;CEK APAKAH BX = 0
        JZ SANA     ;JIKA YA, END
        ADD CH,0    ;CEK APAKAH CH = 0
        JZ SANA     ;JIKA YA,END

        MOV CL,0    ;MENGOSONGKAN WADAH CL
        MOV DX,0    ;MENGOSONGKAN WADAH DH:DL

 SINI:  MOV AL,DL   ;AL <- DL AGAR BISA DIKOREKSI
        ADD AL,BL   ;AL <- AL + BL
        CALL SDAA   ;KOREKSI
        MOV DL,AL   ;DL <- AL KEMBALIKAN

        MOV AL,DH   ;AL <- DH AGAR BISA DIKOREKSI
        ADC AL,BH   ;AL < AL + BH
        CALL SDAA   ;KOREKSI
        MOV DH,AL   ;DH <- AL KEMBALIKAN

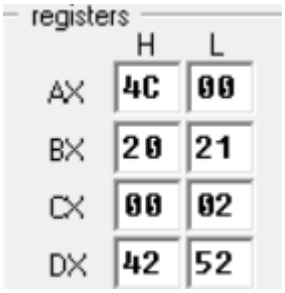
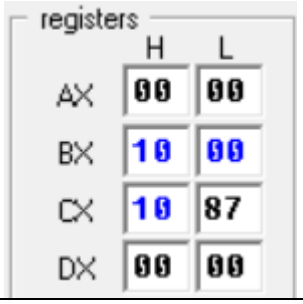
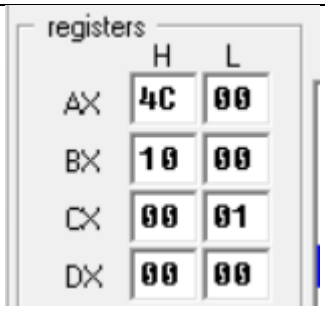
        MOV AL,CL   ;AL <- CL AGAR BISA DIKOREKSI
        ADC AL,0    ;TAMBAHKAN CARRY (AL+CARRY+0)
        CALL SDAA   ;KOREKSI
        MOV CL,AL   ;CL <- AL KEMBALIKAN

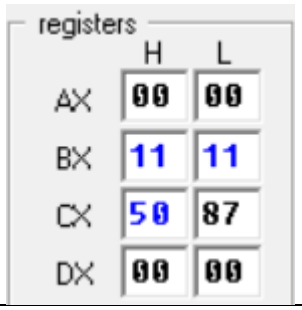
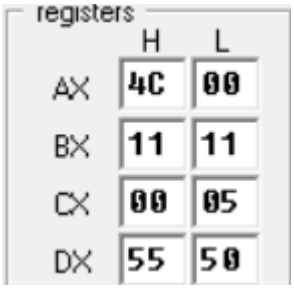
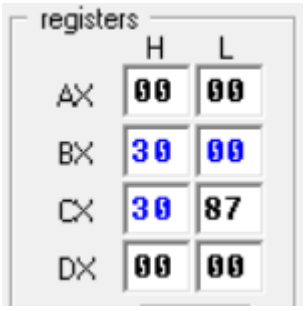
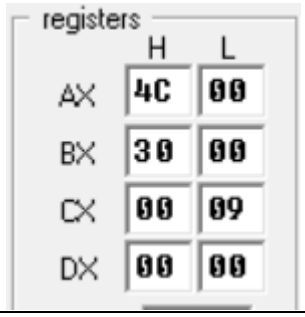
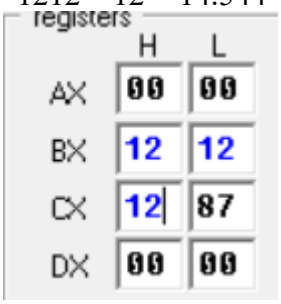
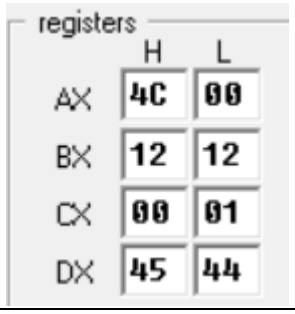
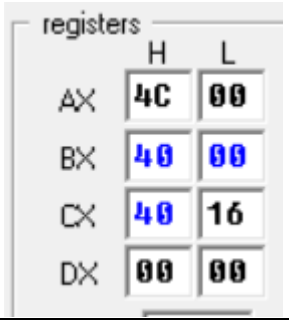
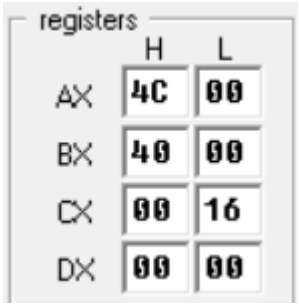
        MOV AL,CH   ;AL <- CH AGAR BISA DAS
        SUB AL,1    ;PENGHITUNG LOOP (HITUNG MUNDUR)
        DAS         ;KOREKSI
        MOV CH,AL   ;CH <- AL KEMBALIKAN
        JNZ SINI    ;LOOP

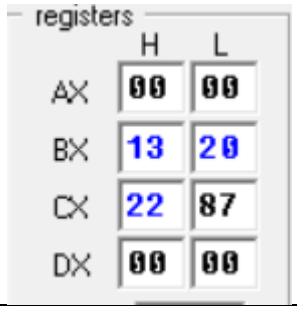
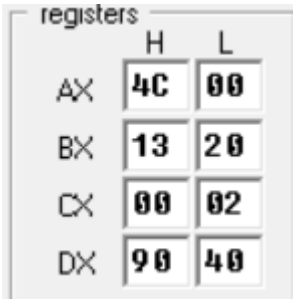
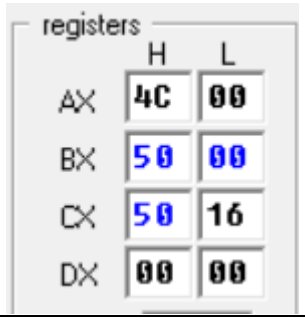
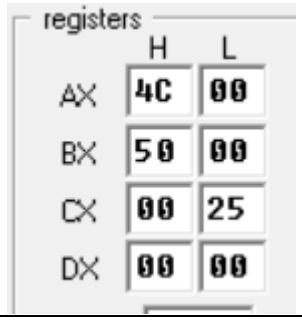
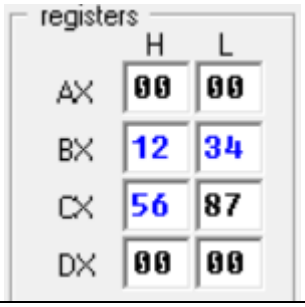
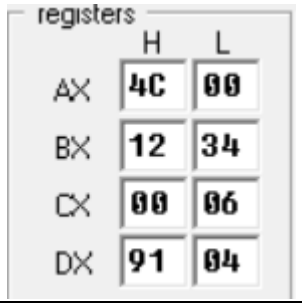
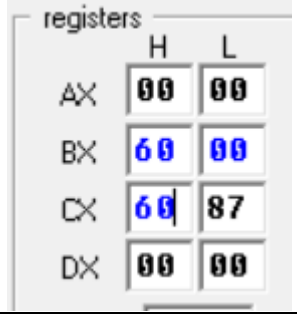
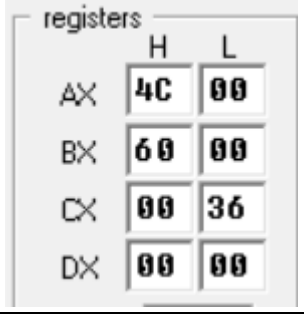
 SANA:  MOV AH,4CH
        INT 21H
        MAIN ENDP
```

```
SDAA PROC   ; SUBRUTIN SDAA
        PUSH BX
        PUSH CX
        MOV BH,AH
        LAHF
        MOV BL,AH
        MOV CH,AL
        MOV CL,AL
        AND CL, 0FH; CL=SATUAN
        ;===============
        TEST AH,10H
        JZ L22
        ADD AL,6
        JNC L22
        ADD AL,60H
        OR BL,1
 L22:   TEST AH,1
        JZ L33
        ADD AL,60H
        OR BL,1
 L33:   MOV CH,AL
        AND CH,0FH
        CMP CH,9
        JBE L44
        ADD AL,6
        JNC L44
        ADD AL,60H
        OR BL,1
 L44:   MOV CH,AL
        AND CH,0F0H
        CMP CH,90H
        JBE L55
        ADD AL,60H
        OR BL,1
 L55:   MOV AH,BL
        SAHF
        POP CX
        POP BX
        MOV AH,BH
        RET
        SDAA ENDP
END MAIN
```

Hasil program:

Data Percobaan	Hasil Percobaan
2021 * 12 = 24.252 	
1000 * 10 = 10.000 	

1111 * 50 = 55.550 	
3000 * 30 = 90.000 	
1212 * 12 = 14.544 	
4000 * 40 = 16.000 	

1320 * 22 = 29.040 	
5000 * 50 = 25.000 	
1234 * 56 = 69.104 	
6000 * 60 = 36.000 	

Analisis Data :

Program pada soal pertama merupakan program assembly untuk mengalikan bilangan desimal 4 digit dengan bilangan desimal 2 digit menggunakan sistem bilangan BCD (Binary Coded Decimal). Program ini dibuat dengan model memori SMALL dan bekerja pada mikroprosesor 8086. Tujuan program adalah melakukan pengujian terhadap 10 data uji, di mana hasil perkalian dari masing-masing pasangan bilangan disimpan dan dikoreksi agar tetap dalam format desimal yang valid. Di awal program, dilakukan inisialisasi register, termasuk pengecekan apakah nilai awal BX (penampung angka 4 digit)

dan CH (penghitung loop) bernilai nol. Register DX dan CX di-nol-kan untuk mengosongkan tempat penyimpanan hasil sementara.

Selanjutnya, bagian utama dari program berada dalam loop berlabel SINI, yang melakukan proses pengalihan dengan menyalin nilai dari DL ke AL, lalu menggandakan nilai tersebut sebanyak 8 kali (menggunakan operasi ADD AL, AL tiga kali) untuk mensimulasikan perkalian dengan digit tertentu. Hasil perkalian ini kemudian dikoreksi menggunakan subrutin SDAA, yang bertugas memastikan hasilnya tetap valid dalam format BCD. Subrutin ini melakukan pengecekan dan penyesuaian terhadap digit satuan dan puluhan dari hasil perkalian agar tidak melebihi batas angka desimal (9). Hal ini penting karena operasi biner biasa bisa menghasilkan nilai lebih dari 9 yang tidak valid dalam representasi BCD.

Setelah koreksi dilakukan, hasil disimpan kembali di register yang sesuai, dan program melanjutkan proses untuk digit berikutnya, termasuk penanganan carry (nilai lebih) yang bisa muncul dari hasil penambahan. Loop ini terus berjalan dengan mengurangi nilai di register CH (penghitung mundur) hingga mencapai nol. Jika sudah selesai, program akan keluar dengan perintah INT 21H, yang merupakan instruksi standar untuk mengakhiri program di sistem DOS.

2. Buat Program untuk mengalikan data desimal 4 digit yang ada pada register BX dengan data desimal 4 digigt yang ada pada register CX. Simpan hasil perkalian pada register SI:DX.

Program :

```
TITLE SOAL 2 ;PERKALIAN 2 DATA DESIMAL (4
.MODEL SMALL
.STACK 100H
.DATA
DAT DB 0,0,0,0
.CODE
MAIN PROC
     MOV AX, @DATA
     MOV DS, AX

     MOV SI, 0
     MOV DX, 0

     CMP CX, 0
     JZ SANA

     CMP BX, 0
     JZ SANA

          ;PENJUMLAHAN SEBANYAK CL KALI
SINI:MOV AL, DL
     ADD AL, BL
     CALL SDAA
     MOV DL, AL

     MOV AL, DH
     ADC AL, BH
     CALL SDAA
     MOV DH, AL

     MOV AL, [DAT]
     ADC AL, 0
     CALL SDAA
     MOV [DAT], AL

     MOV AL, CL
     SUB AL, 1
     DAS
     MOV CL, AL
     JNZ SINI

     MOV [DAT+1], DH
     MOV [DAT+2], DL
     MOV DX, 0          ;PASTIKAN DX KE
```

```
SONO:MOV AL, DL
     ADD AL, BL
     CALL SDAA
     MOV DL, AL

     MOV AL, DH
     ADC AL, BH
     CALL SDAA
     MOV DH, AL

     MOV AL, [DAT+3]
     ADC AL, 0
     CALL SDAA
     MOV [DAT+3], AL

     MOV AL, CH
     SUB AL, 1
     DAS
     MOV CH, AL
     JNZ SONO

     ;PENJUMLAHAN AKHIR
     MOV AL, [DAT+1]
     ADD AL, DL
     CALL SDAA
     MOV [DAT+1], AL

     MOV AL, [DAT]
     ADC AL, DH
     CALL SDAA
     MOV [DAT], AL

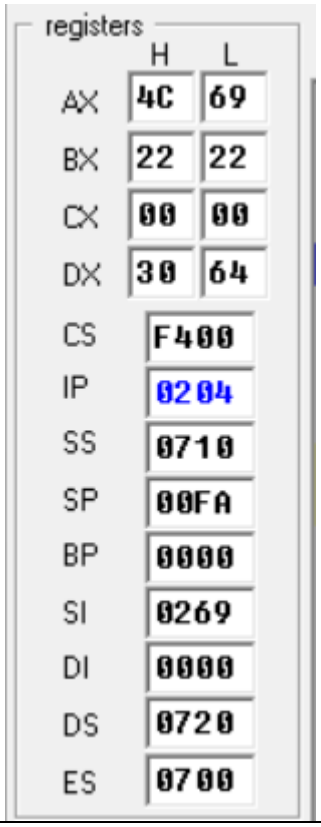
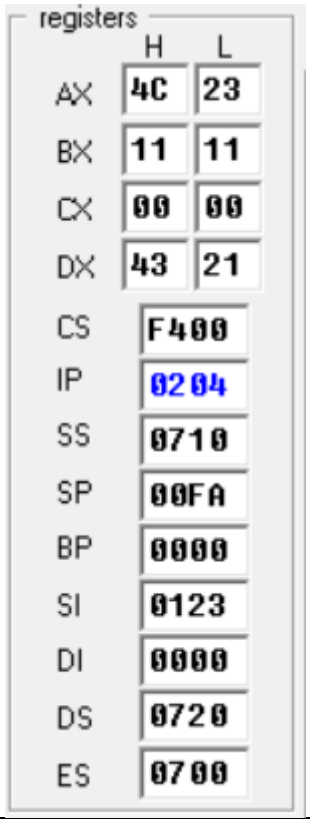
     MOV AL, [DAT+3]
     ADC AL, 0
     CALL SDAA
     MOV [DAT+3], AL

     ;PINDAHKAN KE DI
     MOV AH, [DAT+3]
     MOV AL, [DAT]
     MOV SI, AX

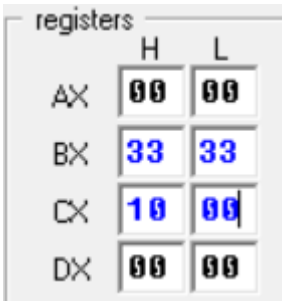
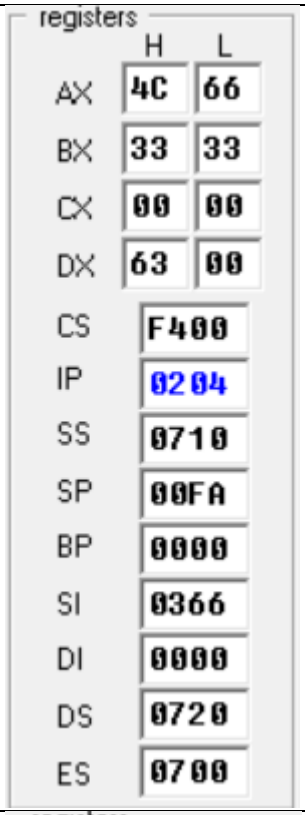
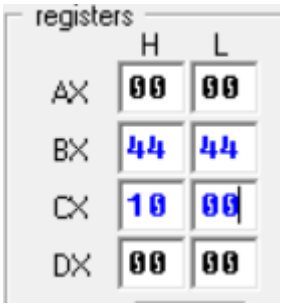
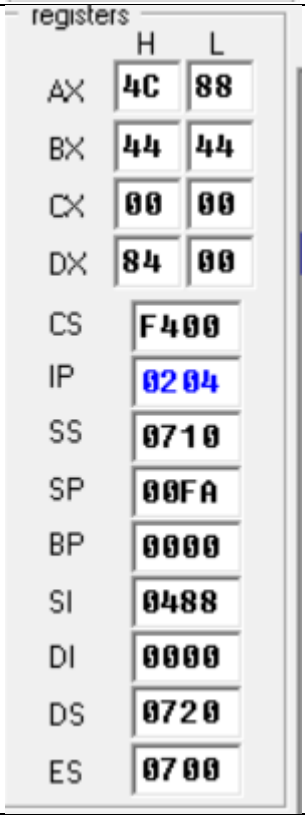
     ;PINDAHKAN KE DX
     MOV DL, [DAT+2]
     MOV DH, [DAT+1]
```

```
SANA:MOV AH, 4CH
     INT 21H
MAIN ENDP
SDAA PROC   ; SUBRUTIN SDAA
     PUSH BX
     PUSH CX
     MOV BH,AH
     LAHF
     MOV BL,AH
     MOV CH,AL
     MOV CL,AL
     AND CL, 0FH; CL=SATUAN
     ;===============
     TEST AH,10H
     JZ L22
     ADD AL,6
     JNC L22
     ADD AL,60H
     OR BL,1
L22: TEST AH,1
     JZ L33
     ADD AL,60H
     OR BL,1
L33: MOV CH,AL
     AND CH,0FH
     CMP CH,9
     JBE L44
     ADD AL,6
     JNC L44
     ADD AL,60H
     OR BL,1
L44: MOV CH,AL
     AND CH,0F0H
     CMP CH,90H
     JBE L55
     ADD AL,60H
     OR BL,1
L55: MOV AH,BL
     SAHF
     POP CX
     POP BX
     MOV AH,BH
     RET
SDAA ENDP
END MAIN
```

Hasil program:

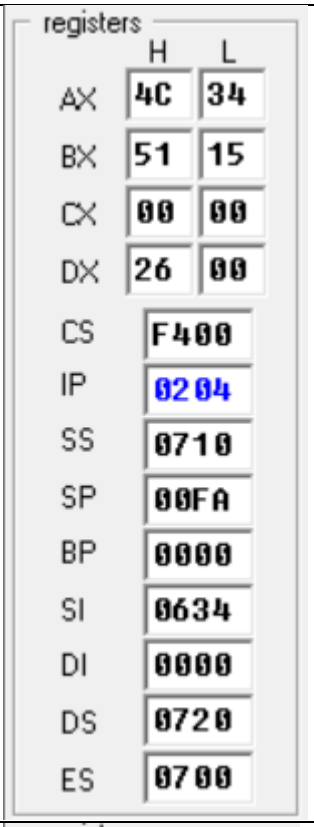
Data Percobaan	Hasil Percobaan
2222 * 1212 = 2.693.064 	
1111 * 1111 = 1.234.321 	

9999 * 1111 = 11.108.889 registers H L AX 00 00 BX 99 99 CX 11 11 DX 00 00	registers H L AX 4C 10 BX 99 99 CX 00 00 DX 88 89 CS F400 IP 0204 SS 0710 SP 00FA BP 0000 SI 1110 DI 0000 DS 0720 ES 0700
2321 * 1205 = 2.796.805 registers H L AX 00 00 BX 23 21 CX 12 05 DX 00 00	registers H L AX 4C 79 BX 23 21 CX 00 00 DX 68 05 CS F400 IP 0204 SS 0710 SP 00FA BP 0000 SI 0279 DI 0000 DS 0720 ES 0700

3333 * 1000 = 3.333.000  	
4444 * 1000 = 4.444.000  	

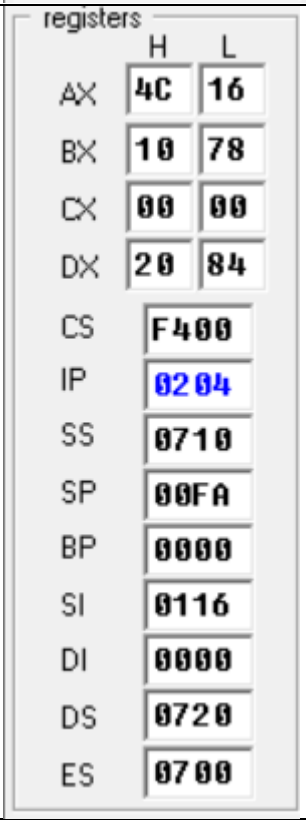
5115 * 1240 = 6.342.600





1078 * 1078 = 1.162.084





1001 * 1002 = 1.003.002 registers H L AX 00 00 BX 10 01 CX 10 02 DX 00 00	registers H L AX 4C 00 BX 10 01 CX 00 00 DX 30 02 CS F400 IP 0204 SS 0710 SP 00FA BP 0000 SI 0100 DI 0000 DS 0720 ES 0700
1010 * 1112 = 1.123.120 registers H L AX 00 00 BX 10 10 CX 11 12 DX 00 00	registers H L AX 4C 12 BX 10 10 CX 00 00 DX 31 20 CS F400 IP 0204 SS 0710 SP 00FA BP 0000 SI 0112 DI 0000 DS 0720 ES 0700

Analisis Data :

Program pada percobaan soal ke-2 yaitu pembuata program untuk melakukan perkalian antara dua bilangan desimal masing-masing 4 digit menggunakan sistem bilangan BCD (Binary Coded Decimal). Prinsip

utama dari program ini adalah melaksanakan proses perkalian melalui metode penjumlahan berulang, di mana salah satu bilangan ditambahkan sebanyak nilai dari bilangan lainnya, sambil tetap menjaga hasil dalam format BCD.

Pada bagian awal program, beberapa register penting diinisialisasi seperti AX, SI, CX, dan DX untuk menyiapkan data dan hasil sementara. Selanjutnya, program memeriksa apakah masih ada data uji yang harus dihitung. Jika tidak ada, program akan lompat ke akhir dan berhenti dengan perintah INT 21H. Jika masih ada data, program masuk ke bagian utama proses perkalian yang ditandai dengan label SINI dan SONO. Di bagian ini, nilai dari dua bilangan yang akan dikalikan dibaca dari memori, kemudian dilakukan proses penjumlahan berulang sebanyak nilai yang ditentukan oleh bilangan pengali. Setiap kali penjumlahan dilakukan, hasilnya dikoreksi menggunakan subrutin SDAA, yaitu prosedur untuk memastikan bahwa setiap digit dari hasil penjumlahan tetap valid dalam format BCD. Koreksi ini penting karena hasil penjumlahan dalam biner bisa menghasilkan digit di luar rentang 0–9.

Proses di dalam SONO juga menangani carry atau sisa hasil penjumlahan yang melebihi satu digit. Setelah satu putaran selesai, nilai hasil disimpan kembali di data memori, dan program melanjutkan ke pasangan bilangan selanjutnya hingga seluruh data uji habis. Pada akhirnya, hasil perkalian yang sudah dikoreksi dipindahkan ke register DI dan DX untuk ditampilkan atau disimpan.

3. Buat program untuk mengalikan data desimal 6 digit yang ada pada register BL:CX dengan data 2 digit yang ada pada register BH. Simpan hasil perkalian pada regsiter DI : DX

Program :

```
TITLE PERKALIAN NO3
.MODEL SMALL
.STACK 100H
.DATA
DAT DB 0,0
.CODE
MAIN PROC
     MOV AX, @DATA
     MOV DS, AX
     MOV DI, 0
     MOV DX, 0
     CMP BH,0
     JZ SANA
     CMP CX, 0
     JZ SANA
     CMP BL, 0
     JZ SANA

SINI:MOV AL, DL
     ADD AL, CL
     CALL SDAA
     MOV DL, AL

     MOV AL, DH
     ADC AL, CH
     CALL SDAA
     MOV DH, AL

     MOV AL, [DAT]
     ADC AL, BL
     CALL SDAA
     MOV [DAT], AL

     MOV AL, [DAT+1]
     ADC AL, 0
     CALL SDAA
     MOV [DAT+1], AL

     MOV AL, [DAT]
     MOV AH, [DAT+1]
     MOV DI, AX

     MOV AL, [DAT+1]
     ADC AL, 0
     CALL SDAA
     MOV [DAT+1], AL

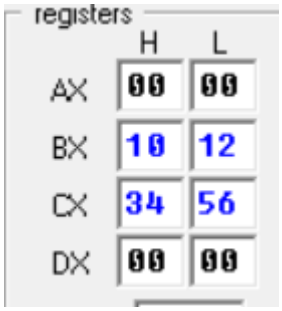
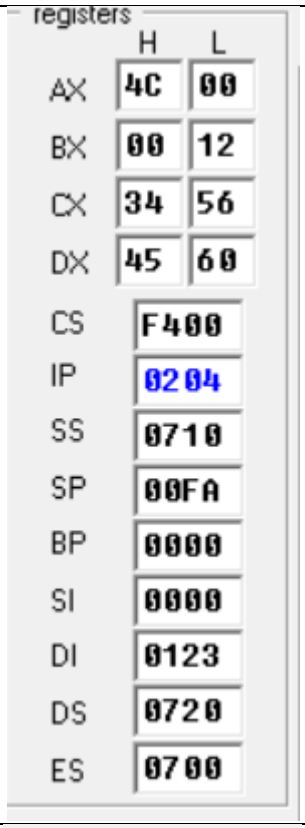
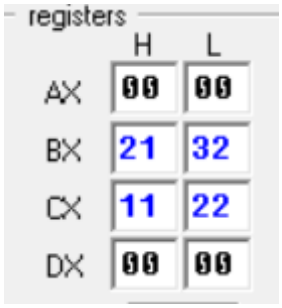
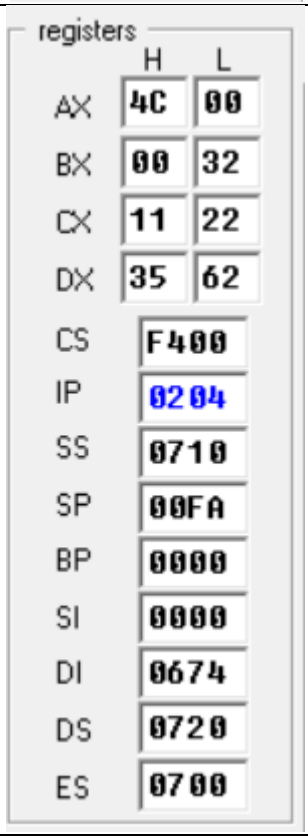
     MOV AL, [DAT]
     MOV AH, [DAT+1]
     MOV DI, AX

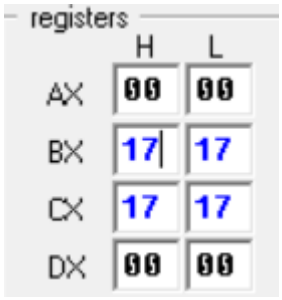
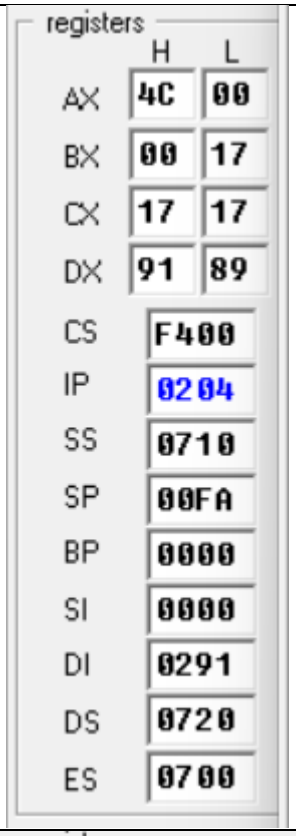
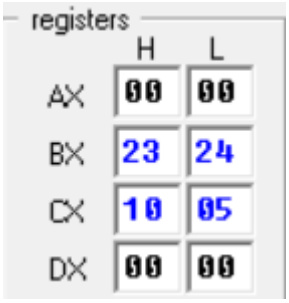
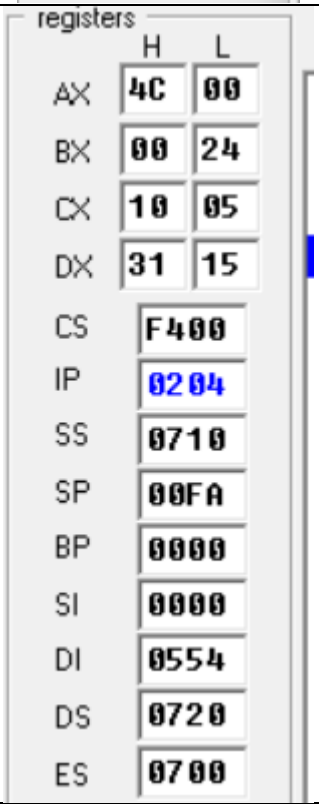
     MOV AL, BH
     SUB AL, 1
     DAS
     MOV BH, AL
     JNZ SINI

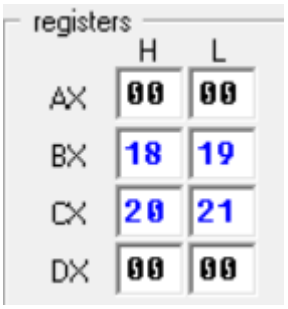
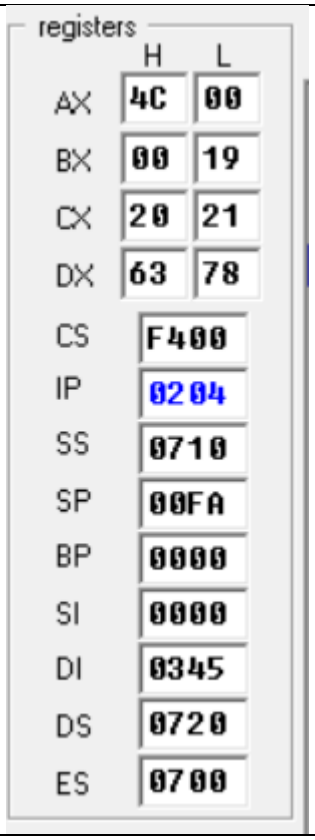
SANA:MOV AH, 4CH
     INT 21H
MAIN ENDP

SDAA PROC   ; SUBRUTIN SDAA
     PUSH BX
     PUSH CX
     MOV BH,AH
     LAHF
     MOV BL,AH
     MOV CH,AL
     MOV CL,AL
     AND CL, 0FH; CL=SATUAN
     ;===============
     TEST AH,10H
     JZ L22
     ADD AL,6
     JNC L22
     ADD AL,60H
     OR BL,1
L22: TEST AH,1
     JZ L33
     ADD AL,60H
     OR BL,1
L33: MOV CH,AL
     AND CH,0FH
     CMP CH,9
     JBE L44
     ADD AL,6
     JNC L44
     ADD AL,60H
     OR BL,1
L44: MOV CH,AL
     AND CH,0F0H
     CMP CH,90H
     JBE L55
     ADD AL,60H
     OR BL,1
L55: MOV AH,BL
     SAHF
     POP CX
     POP BX
     MOV AH,BH
     RET
SDAA ENDP
END MAIN
```

Hasil program:

Data Percobaan	Hasil Percobaan
123456 * 10 = 1.234.560 	
321122 * 21 = 6.743.562 	

171717 * 17 = 2.919.189 	
241005 * 23 = 5.543.115 	

192021 * 18 = 3.456.378 	

Analisis Data :

Pada soal No.-3 diperintahkan untuk membuat program yang dapat mengalikan bilangan desimal 6 digit yang disimpan dalam register BL:CX (BL = digit tertinggi, CX = digit rendah) dengan bilangan 2 digit yang disimpan di register BH. Hasil perkalian disimpan pada register DI:DX, yang berarti hasil akhir dapat memiliki hingga 8 digit desimal.

Langkah pertama, program memeriksa apakah nilai pada BH, CX, atau BL adalah nol. Jika iya, maka proses dihentikan karena hasil pasti nol. Jika semua nilai valid, maka program melanjutkan proses perkalian dengan cara penjumlahan berulang, yaitu menjumlahkan isi dari CX (bilangan rendah) sebanyak BH kali. Setiap hasil penjumlahan dikoreksi dengan subrutin SDAA agar tetap dalam format desimal (BCD).

Kemudian, hasil parsial dari perkalian disimpan ke dalam DX dan dilanjutkan dengan memperhitungkan bagian atas dari bilangan 6 digit (BL). Perulangan dilakukan hingga semua digit dari pengali (BH) habis digunakan. Akhirnya, hasil total disimpan dalam register DX (bagian bawah) dan DI (bagian atas), membentuk hasil 8 digit desimal.

Subrutin SDAA berfungsi mengoreksi setiap hasil operasi penjumlahan agar valid dalam format BCD. Koreksi ini penting karena operasi aritmatika pada prosesor dilakukan dalam biner, bukan desimal.

Secara singkat, program ini merupakan implementasi manual dari perkalian desimal panjang (6 digit × 2 digit) menggunakan penjumlahan berulang dan koreksi BCD, dengan hasil disimpan dalam dua register utama (DI:DX).

4.Buat program untuk mengalikan data desimal 4 digit yang ada pada register BX dengan data desimal 6 digit yang ada pada register CH:DX. Simpan hasil perkalian pada data area[10 digit]

Program :

```
TITLE N04
.MODEL SMALL
.STACK 100H
.DATA
DAT DB 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
HSL DB 0,0,0,0,0,0
.CODE
MAIN PROC
     MOV AX, @DATA
     MOV DS, AX
     CMP CX, 0
     JZ SANA
     CMP BX, 0
     JZ SANA

  SINI:  MOV AL, [DAT]
         ADD AL, BL
         CALL SDAA
         MOV [DAT], AL

         MOV AL, [DAT+1]
         ADC AL, BH
         CALL SDAA
         MOV [DAT+1], AL

         MOV AL, [DAT+2]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+2], AL

         MOV AL, DL
         SUB AL, 1
         DAS
         MOV DL, AL
         JNZ SINI

    HAI:MOV AL, [DAT+3]
         ADD AL, BL
         CALL SDAA
         MOV [DAT+3], AL

         MOV AL, [DAT+4]
         ADC AL, BH
         CALL SDAA
         MOV [DAT+4], AL

         MOV AL, [DAT+5]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+5], AL

         MOV AL, DH
         SUB AL, 1
         DAS
         MOV DH, AL
         JNZ HAI

    HOI:MOV AL, [DAT+6]
         ADD AL, BL
         CALL SDAA
         MOV [DAT+6], AL

         MOV AL, [DAT+7]
         ADC AL, BH
         CALL SDAA
         MOV [DAT+7], AL

         MOV AL, [DAT+8]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+8], AL

         MOV AL, CH
         SUB AL, 1
         DAS
         MOV CH, AL
         JNZ HOI

         MOV AL, [DAT]
         MOV [HSL+4], AL

         MOV AL, [DAT+1]
         ADC AL, [DAT+3]
         CALL SDAA
         MOV [HSL+3], AL

         MOV AL, [DAT+2]
         ADC AL, [DAT+4]
         CALL SDAA
         MOV [DAT+4], AL

         MOV AL, [DAT+9]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+9], AL

         MOV AL, [DAT+4]
         ADD AL, [DAT+6]
         CALL SDAA
         MOV [HSL+2], AL

         MOV AL, [DAT+9]
         ADC AL, [DAT+5]
         CALL SDAA
         MOV [DAT+5], AL

         MOV AL, [DAT+10]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+10], AL

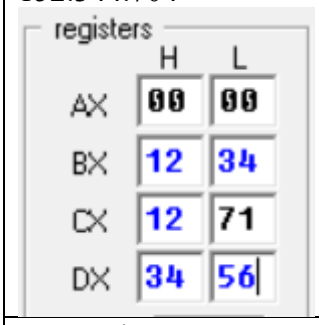
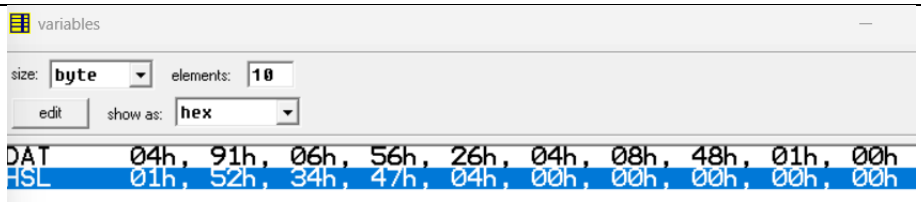
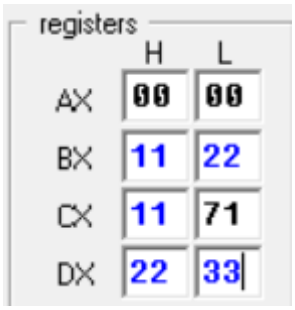
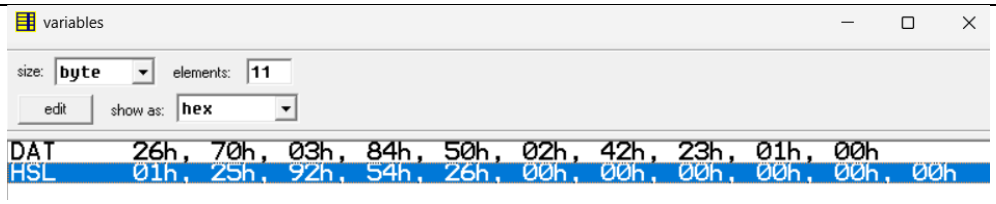
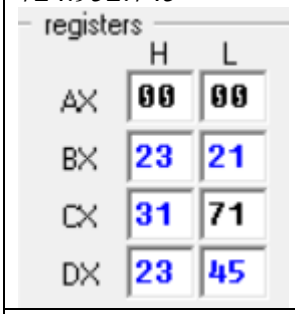
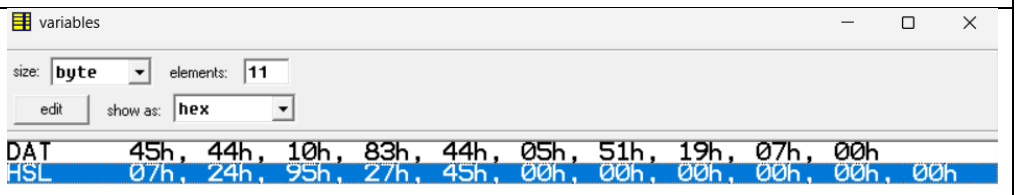
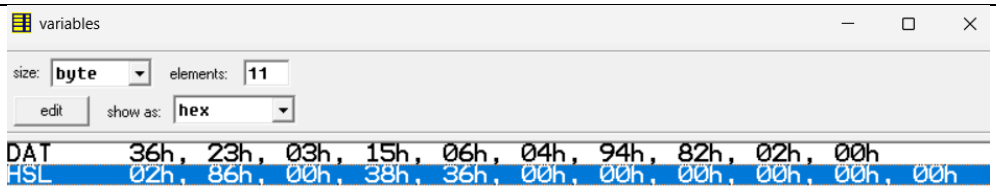
         MOV AL, [DAT+5]
         ADD AL, [DAT+7]
         CALL SDAA
         MOV [HSL+1], AL

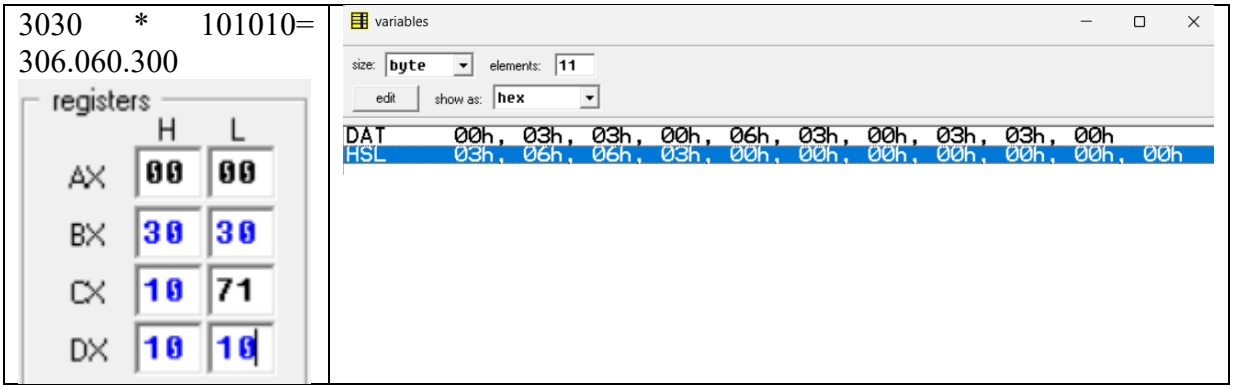
         MOV AL, [DAT+8]
         ADC AL, [DAT+10]
         CALL SDAA
         MOV [HSL], AL

SANA:MOV AH, 4CH
     INT 21H
MAIN ENDP
```

```
SDAA PROC   ; SUBRUTIN SDAA
    PUSH BX
    PUSH CX
    MOV BH,AH
    LAHF
    MOV BL,AH
    MOV CH,AL
    MOV CL,AL
    AND CL, 0FH; CL=SATUAN
    ;===============
    TEST AH,10H
    JZ L22
    ADD AL,6
    JNC L22
    ADD AL,60H
    OR BL,1
L22: TEST AH,1
    JZ L33
    ADD AL,60H
    OR BL,1
L33: MOV CH,AL
    AND CH,0FH
    CMP CH,9
    JBE L44
    ADD AL,6
    JNC L44
    ADD AL,60H
    OR BL,1
L44: MOV CH,AL
    AND CH,0F0H
    CMP CH,90H
    JBE L55
    ADD AL,60H
    OR BL,1
L55: MOV AH,BL
    SAHF
    POP CX
    POP BX
    MOV AH,BH
    RET
SDAA ENDP
END MAIN
```

Hasil program:

Data Percobaan	Hasil Percobaan
1234 * 123456 = 152.344.704 	
1122 * 112233 = 125.925.426 	
2321 * 312345 = 724.952.745 	
2021 * 141516 = 286.003.836 	



Analisis Data :

Program Assembly ini dirancang untuk mengalikan dua bilangan desimal, yaitu bilangan 4 digit yang tersimpan dalam register BX dan bilangan 6 digit yang tersimpan dalam register CH:DX. Hasil perkalian tersebut disimpan dalam area data HSL sebanyak 10 digit. Program ini tidak menggunakan instruksi perkalian langsung seperti MUL, melainkan menggunakan metode perkalian manual berbasis desimal (BCD), di mana setiap digit bilangan dikalikan satu per satu, kemudian hasilnya dijumlahkan dan disesuaikan menggunakan carry (ADC). Proses ini melibatkan penggunaan instruksi MOV untuk memindahkan data antar register atau memori, ADD dan ADC untuk penjumlahan digit dan carry, SUB dan DAS untuk koreksi hasil, serta instruksi logika dan kontrol seperti CMP, JZ, dan JNZ untuk pengendalian alur program. Selain itu, program menggunakan subrutin SDAA untuk memastikan bahwa setiap hasil operasi tetap dalam format BCD yang benar, dengan memanfaatkan instruksi seperti AND, TEST, OR, dan ADD untuk menyesuaikan digit satuan dan puluhan. Subrutin ini juga menjaga nilai register dengan PUSH dan POP agar proses koreksi tidak merusak data utama. Setelah proses pengalihan selesai, hasil akhir disimpan ke dalam array HSL secara berurutan.

5.Buat program untuk mengalikan data desimal 6 digit yang ada pada register CL : BX dengan data desimal 6 digit yang ada pada register CH:DX. Simpan hasil perkalian pada data area[12 digit]

Program :

```
TITLE SOAL 5 ;[6 * 6 DIGIT DESIMAL]
.MODEL SMALL
.STACK 100H
.DATA
DAT DB 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
HSL DB 0,0,0,0,0,0
.CODE
MAIN PROC
     MOV AX, @DATA
     MOV DS, AX

     CMP CX, 0
     JZ SANA

     CMP BX, 0
     JZ SANA

  SINI:  MOV AL, [DAT]
         ADD AL, BL
         CALL SDAA
         MOV [DAT], AL

         MOV AL, [DAT+1]
         ADC AL, BH
         CALL SDAA
         MOV [DAT+1], AL

         MOV AL, [DAT+2]
         ADC AL, CL
         CALL SDAA
         MOV [DAT+2], AL

         MOV AL, [DAT+3]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+3], AL

         MOV AL, DL
         SUB AL, 1
         DAS
         MOV DL, AL
         JNZ SINI

         ;PENJUMLAHAN SEBANYAK DH KALI
```

```
    SONO:MOV AL, [DAT+4]
         ADD AL, BL
         CALL SDAA
         MOV [DAT+4], AL

         MOV AL, [DAT+5]
         ADC AL, BH
         CALL SDAA
         MOV [DAT+5], AL

         MOV AL, [DAT+6]
         ADC AL, CL
         CALL SDAA
         MOV [DAT+6], AL


         MOV AL, [DAT+7]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+7], AL

         MOV AL, DH
         SUB AL, 1
         DAS
         MOV DH, AL
         JNZ SONO

    SUNU:MOV AL, [DAT+8]
         ADD AL, BL
         CALL SDAA
         MOV [DAT+8], AL

         MOV AL, [DAT+9]
         ADC AL, BH
         CALL SDAA
         MOV [DAT+9], AL

         MOV AL, [DAT+10]
         ADC AL, CL
         CALL SDAA
         MOV [DAT+10], AL

         MOV AL, [DAT+11]
         ADC AL, 0
         CALL SDAA
```

```
         MOV [DAT+11], AL

         MOV AL, CH
         SUB AL, 1
         DAS
         MOV CH, AL
         JNZ SUNU

         MOV AL, [DAT+0]
         MOV [HSL+5], AL   ;HSL+5

         MOV AL, [DAT+1]
         ADC AL, [DAT+4]
         CALL SDAA
         MOV [HSL+4], AL   ;HSL+4

         MOV AL, [DAT+2]
         ADC AL, [DAT+5]
         CALL SDAA
         MOV [DAT+5], AL

         MOV AL, [DAT+12]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+12], AL

         MOV AL, [DAT+5]
         ADD AL, [DAT+8]
         CALL SDAA
         MOV [HSL+3], AL   ;HSL+3

         MOV AL, [DAT+3]
         ADC AL, [DAT+12]
         CALL SDAA
         MOV [DAT+3], AL

         MOV AL, [DAT+13]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+13], AL

         MOV AL, [DAT+3]
         ADD AL, [DAT+6]
         CALL SDAA
```

```
         MOV AL, [DAT+13]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+13], AL

         MOV AL, [DAT+9]
         ADD AL, [DAT+6]
         CALL SDAA
         MOV [HSL+2], AL    ;HSL+2

         MOV AL, [DAT+7]
         ADC AL, [DAT+13]
         CALL SDAA
         MOV [DAT+7], AL

         MOV AL, [DAT+14]
         ADC AL, 0
         CALL SDAA
         MOV [DAT+14], AL

         MOV AL, [DAT+7]
         ADD AL, [DAT+10]
         CALL SDAA
         MOV [HSL+1], AL    ;HSL+1

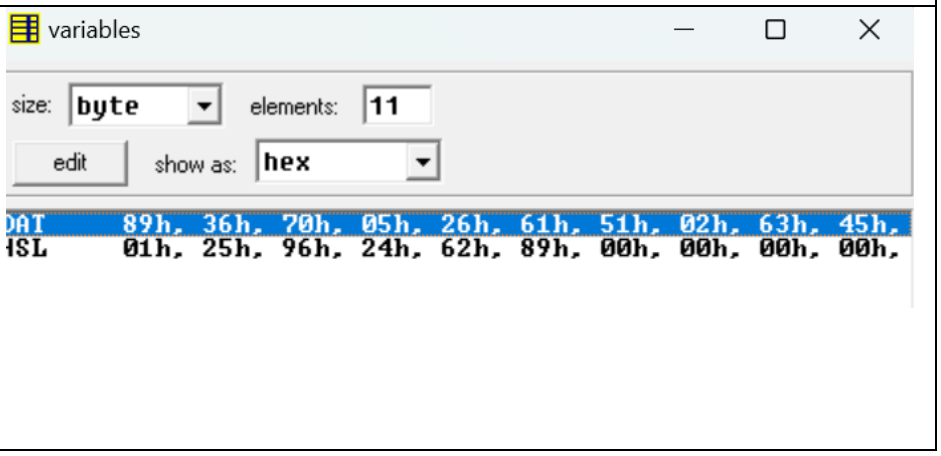
         MOV AL, [DAT+11]
         ADC AL, [DAT+14]
         CALL SDAA
         MOV [DAT+11], AL

         MOV AL, [DAT+11]
         MOV [HSL], AL

         SANA:MOV AH, 4CH
         INT 21H
MAIN ENDP
```

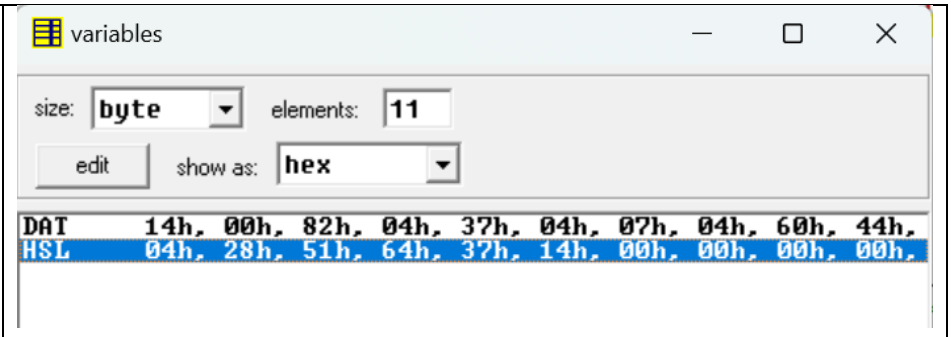
```
SDAA PROC    ; SUBRUTIN SDAA
    PUSH BX
    PUSH CX
    MOV BH,AH
    LAHF
    MOV BL,AH
    MOV CH,AL
    MOV CL,AL
    AND CL, 0FH; CL=SATUAN
    ;===============
    TEST AH,10H
    JZ L22
    ADD AL,6
    JNC L22
    ADD AL,60H
    OR BL,1
L22: TEST AH,1
    JZ L33
    ADD AL,60H
    OR BL,1
L33: MOV CH,AL
    AND CH,0FH
    CMP CH,9
    JBE L44
    ADD AL,6
    JNC L44
    ADD AL,60H
    OR BL,1
L44: MOV CH,AL
    AND CH,0F0H
    CMP CH,90H
    JBE L55
    ADD AL,60H
    OR BL,1
L55: MOV AH,BL
    SAHF
    POP CX
    POP BX
    MOV AH,BH
    RET
SDAA ENDP
END MAIN
```

Hasil Program :

Data Percobaan	Hasil Percobaan
112233 * 112233 = 12596246289	

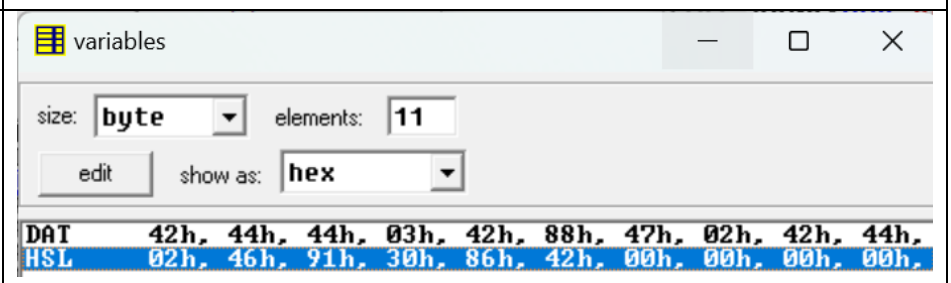
212223 * 201918= 42.851.643.714



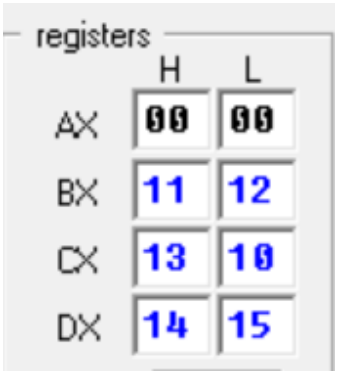


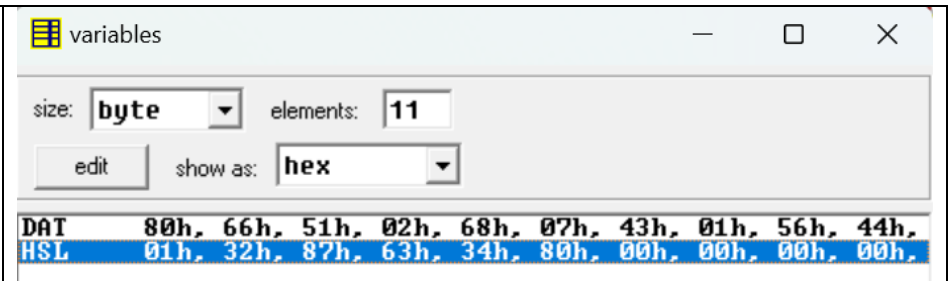
222222 * 111111 = 24.691.308.642





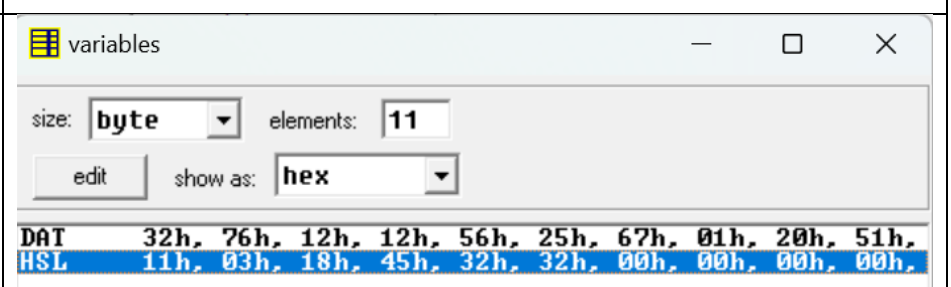
101112 * 131415 = 13.287.633.480





551256 * 200122 = 110.318.453.232





Analisis Data :

Pada percobaan soal ke-5 yang merupakan program Assembly di atas merupakan implementasi dari operasi perkalian dua bilangan desimal 6 digit yang masing-masing disimpan dalam register CL:BX dan CH:DX, dengan hasil perkalian disimpan sebagai 12 digit desimal di area data. Proses perkalian dilakukan dengan pendekatan penjumlahan berulang (repeated addition), yang dibagi menjadi tiga tahap sesuai digit-digit penyusun bilangan pengali: tahap pertama mengalikan dengan DL, tahap kedua dengan DH, dan tahap ketiga dengan CH. Masing-masing hasil penjumlahan dari ketiga tahap disimpan dalam bagian berbeda dari array DAT, yaitu dari DAT[0] hingga DAT[11]. Setelah ketiga tahap penjumlahan selesai, program melanjutkan dengan menjumlahkan hasil-hasil parsial tersebut untuk menghasilkan nilai akhir perkalian 12 digit. Agar format desimal tetap valid, digunakan subrutin SDAA (Software Decimal Adjust for Addition) setiap kali terjadi penjumlahan atau propagasi carry, yang memastikan nilai tetap dalam format BCD (Binary Coded Decimal). Akhirnya, hasil akhir disimpan dalam array HSL, mewakili digit-digit paling signifikan hingga paling rendah dari hasil perkalian. Program ini meniru metode perkalian manual seperti pada kertas, namun dilakukan secara terstruktur di tingkat instruksi Assembly.

F. KESIMPULAN

Pada praktikum ini dapat disimpulkan bahwa praktikan dapat lebih memahami cara kerja software EMU8086 yang dapat menjalankan program yang digunakan untuk mikroprosesor. Praktikan juga lebih memahami bagaimana cara membuat program untuk mengalikan data pada bilangan desimal yang akan ditampilkan dalam bentuk BCD pada monitor. Operasi perkalian dilakukan menggunakan penggunaan metode penjumlahan berulang, sehingga perintah yang digunakan adalah perintah operasi penjumlahan pada operasi bilangan desimal. Penggunaan peintah DAA dan subrutin SDAA sangat membantu praktikan untuk menjalankan program penjumlahan dalam bentuk desimal mendapatkan nilai yang sesuai dengan perhitungan. Dan pada praktikum ini juga, praktikan mengetahui bagaimana hasil penjumlahan pada program dapat disimpan dan ditampilkan pada memory(vars) ataupun pada register juga di software EMU8086.

DAFTAR PUSTAKA

Arief Wisaksono, I. M. (2019). *Buku Ajar Mikroprosesor* . Retrieved from eprint.umsida.ac.id:http://eprints.umsida.ac.id/7199/1/31.%20buku%20ajar%20mikroprosessor%20komplate%20-%20Arief%20Wisaksono.pdf

Hasanah, Rahmawati. 2019. Mikroprosesor dan Interface. Bandung: Politeknik Negeri Bandung

S.to. 2001. “Pemrograman Dengan Bahasa Assembly Edisi Online Versi 1.0